

Luiz
Pedro Augusto dos Santos Kzan 12547465

Relatório 6 - Projetos em Sistemas Embarcados

Prof. Dr. Pedro de Oliveira Conceição Júnior

Parte 1:

Produto escolhido: **WS-IP316-204-A1** - Câmera de segurança IP genérica com SoC Hi3516C V300

Descrição: Câmera de segurança IP, envia as imagens de vídeo capturadas e gravações por uma rede ip, é a prova d'água.

1. Unidade de Processamento

- Tipo de unidade de processamento utilizada: ARM926; SoC: Hi3516C V300
- Plataforma de integração: placa dedicada com SoC Hi3516C V300 que contém memória DDR externa, memória Flash, sensor CMOS, PHY Ethernet e demais periféricos necessários à aplicação
- Fabricante: WINSAFE/OEM e HiSilicon
- Arquitetura: ARM926 -> arquitetura ARM9
- Frequência de clock: 800MHz

2. Memória

- Tipo e capacidade de RAM: SDRAM 16bit DDR3/ DDR3L; capacidade de 4GB
- Tipo e capacidade de Flash/ROM: SPI NOR flash interface – 1-/2-/4-wire mode – Maximum capacity of 32 MB;

SPI NAND flash interface with maximum 4 Gbits capacity
- Presença e tipo de EEPROM (interna ou externa): não citado, não deve ter

3. Sistema Operacional

- Tipo de sistema operacional: Linux; biblioteca de decodificação H.264 de PC; biblioteca de decodificação H.265 de PC, Android e iOS
- Versão ou kernel, se aplicável: Linux 3.18

4. Interfaces de Comunicação com Fio

- Interfaces identificadas: (I2C, SPI, UART, CAN, Ethernet, USB etc.)

I2Cx2

UARTx3

SPI / SSP (SSPx3)

Ethernet (ETH – RMII 10/100 Mbps)

USB 2.0 (Host/Device)

SDIO 2.0

Flash Interface (SPI Flash / SPI NAND)

DDRC (DDR3 / DDR3L)

5. Interfaces de Comunicação sem Fio

- Tecnologias presentes: Feito de forma externa ao SoC, ou seja, pela fabricante que compra o SoC. Suporta IP estático, WiFi.

6. Entradas e Saídas (I/O)

- GPIOs disponíveis: Sim, não especificada a quantidade
 - Presença de ADC e/ou DAC: tem ADC, 3 canais LSADC
 - Sinais PWM disponíveis: Sim, 4 canais PWM.
-

7. Sensores e Atuadores

Sensores

- Sensores presentes no dispositivo (tipo e função):

Sensor de imagem CMOS (externo)

Função: captura de imagens/vídeo.

Componente fotossensível (externo, opcional)

Função: detecção de luminosidade ambiente para controle de modo dia/noite (ex.: IR cut).

Atuadores

LEDs infravermelhos (externos)

Função: iluminação noturna para captura em baixa luminosidade.

Atuadores externos de movimento (opcional)

Função: controle de pan/tilt (PTZ), quando existente no produto final.

Interface com o processador

- **Sensores de imagem:** MIPI, LVDS, HiSPi, BT.1120, RGB Bayer
 - **Sensores simples / fotossensível:** GPIO, ADC (LSADC)
 - **Atuadores (LEDs, IR, motores externos):** GPIO, PWM, UART / RS485 (dependendo do projeto)
-

8. Fonte de Energia e Alimentação

- Tipo de alimentação (bateria, fonte externa etc.): não especificado, mas para modelos parecidos é comum 12V ou 5V DC, convertidos da tomada, ou vindo de um soquete de lâmpada. Alimentação externa.
- Tecnologia da bateria, se houver: n/a
- Módulos ou circuitos de gerenciamento de energia (PMIC): POR (Power-On Reset); RTC integrado, Standby wakeup circuit
- Mecanismos de reset e proteção:

POR (Power-On Reset)

Secure Boot

OTP (One-Time Programmable) storage

Gerador de números aleatórios (TRNG)

9. Firmware e Atualizações

- Tipo de firmware utilizado: Linux-3.18-based SDK

- Presença de bootloader: Sim, Suporte a boot a partir de **SPI NOR Flash, SPI NAND Flash** ou **eMMC**; e presença de **Secure Boot**
 - Suporte a atualizações (locais, via rede, OTA etc.): **Atualizações locais ou via rede**, dependentes da implementação do firmware da câmera IP
-

10. Segurança e Proteção

- Recursos de criptografia (hardware ou software):

AES

DES / 3DES

RSA

HASH (SHA1, SHA256, HMAC)

- Mecanismos de autenticação e controle de acesso:

Secure Boot

OTP storage (512 bits)

TRNG (True Random Number Generator)

11. Armazenamento Externo

- Tipos de armazenamento externo suportados: (SD, NAND Flash, HDD, SSD etc.)

SPI NOR Flash (até 32 MB)

SPI NAND Flash (até 4 Gbits)

eMMC 4.5

Cartão SD / SDHC / SDXC (via SDIO) – até 2 TB

12. Interface com o Usuário

- Elementos de interação disponíveis: (display, LEDs, botões, sensores de toque etc.)

LEDs indicadores (externos, controlados por GPIO/PWM)

Display LCD via saída RGB565

Interface IR

Interface CVBS / BT.656 (saída de vídeo)

Alarmes externos (via GPIO / UART)

Realizar uma pesquisa com base em artigos científicos publicados em periódicos revisados por pares e indexados em bases reconhecidas. Dar preferência para a base IEEE Xplore. (ver também: Dicas de Pesquisa e como usar a IEEE Xplore - Como Habilitar Conexão Remota VPN para acesso às bases de dados). No entanto, outras bases eventualmente poderão ser utilizadas (Web of Science, Scopus, ScienceDirect etc.). Deve-se selecionar: • 1 artigo científico que detalha tecnologias centrais que viabilizam o funcionamento do produto (ex.: arquitetura de processador, protocolos de comunicação, sensores específicos, sistemas operacionais embarcados etc.). • 1 artigo científico que apresenta aplicações ou estudos de caso do produto ou de dispositivos similares (ex.: uso em ambiente industrial, doméstico, médico, automotivo, etc.). Para cada artigo, incluir: • Referência completa do artigo (em formato IEEE ou ABNT). Exemplo: Base: IEEE Xplore Termos de busca: ALL=(fault diagnosis) AND ALL=(rotating machinery) Refinamento: ano de publicação = 2020 Referência: S. Tang, S. Yuan and Y. Zhu, "Deep Learning-Based Intelligent Fault Diagnosis Methods Toward Rotating Machinery," IEEE Access, vol. 8, pp. 9335-9346, 2020. doi: 10.1109/ACCESS.2019.2963092 • Resumo de até 10 linhas explicando brevemente o conteúdo geral do estudo. • Análise crítica, respondendo às seguintes perguntas: ○ Qual é a questão de pesquisa principal abordada pelo artigo? Qual a importância do problema tratado para a área do conhecimento?

3 ○ Os autores mencionam limitações do estudo? Se sim, quais? Que pergunta ou crítica você faria aos autores para esclarecer ou aprofundar algum ponto?

Realizar uma pesquisa com base em artigos científicos publicados em periódicos revisados por pares e indexados em bases reconhecidas. Dar preferência para a base IEEE Xplore. No entanto, outras bases eventualmente poderão ser utilizadas, como Web of Science, Scopus, ScienceDirect, etc. Deve-se selecionar:

1. Um artigo científico que detalha tecnologias centrais que viabilizam o funcionamento do produto (por exemplo: arquitetura de processador, protocolos de comunicação, sensores específicos, sistemas operacionais embarcados etc.).
2. Um artigo científico que apresenta aplicações ou estudos de caso do produto ou de dispositivos similares (por exemplo: uso em ambiente industrial, doméstico, médico, automotivo etc.).

Para cada artigo, incluir:

- **Referência completa do artigo** (em formato IEEE ou ABNT).
Exemplo: Base: IEEE Xplore
Termos de busca: ALL=(fault diagnosis) AND ALL=(rotating machinery)
Refinamento: ano de publicação = 2020
Referência: S. Tang, S. Yuan and Y. Zhu, "Deep Learning-Based Intelligent Fault Diagnosis Methods Toward Rotating Machinery," IEEE Access, vol. 8, pp. 9335-9346, 2020. doi: 10.1109/ACCESS.2019.2963092
- **Resumo de até 10 linhas**, explicando brevemente o conteúdo geral do estudo.
- **Análise crítica**, respondendo às seguintes perguntas:
 - Qual é a questão de pesquisa principal abordada pelo artigo? Qual a importância do problema tratado para a área do conhecimento?
 - Os autores mencionam limitações do estudo? Se sim, quais? Que pergunta ou crítica você faria aos autores para esclarecer ou aprofundar algum ponto?

1. Um artigo científico que detalha tecnologias centrais que viabilizam o funcionamento do produto (por exemplo: arquitetura de processador, protocolos de comunicação, sensores específicos, sistemas operacionais embarcados etc.):

B. A. Alpatov, P. V. Babayan and M. D. Ershov, "Embedded Image Processing and Video Analysis in Intelligent Camera-based Vision System," *2020 9th Mediterranean Conference on Embedded Computing (MECO)*, Budva, Montenegro, 2020, pp. 1-4, doi: 10.1109/MECO49872.2020.9134254.

Resumo: O artigo apresenta o desenvolvimento e a implementação de algoritmos de processamento de imagens e análise de vídeo em câmeras inteligentes com arquitetura embarcada. Os autores descrevem as características técnicas de câmeras IP da Axis e discutem os desafios associados à execução de algoritmos de visão computacional em plataformas com recursos computacionais limitados. São analisadas diferentes arquiteturas de processadores embarcados, como MIPS e ARM, bem como suas implicações no desempenho em tempo real. O estudo também avalia técnicas de otimização, como paralelização, uso de aritmética em ponto fixo e redução da resolução das imagens, visando garantir processamento em tempo real. Resultados experimentais mostram o impacto dessas estratégias no tempo de execução de algoritmos de detecção e rastreamento de veículos.

Perguntas:

- Qual é a questão de pesquisa principal abordada pelo artigo? Qual a importância do problema tratado para a área do conhecimento?

R: A principal questão de pesquisa abordada pelo artigo é **como implementar algoritmos de processamento de imagem e análise de vídeo em tempo real em plataformas embarcadas de câmeras inteligentes, considerando suas limitações de hardware**. O problema é altamente relevante para a área de sistemas embarcados e visão computacional, pois a crescente demanda por sistemas de vigilância inteligentes exige que parte significativa do processamento seja realizada localmente, reduzindo o uso de largura de banda e a dependência de servidores centrais. O estudo contribui ao demonstrar, de forma prática, como características da arquitetura do processador influenciam diretamente a viabilidade de algoritmos de visão em aplicações reais.

- Os autores mencionam limitações do estudo? Se sim, quais? Que pergunta ou crítica você faria aos autores para esclarecer ou aprofundar algum ponto?

R: Os autores reconhecem implicitamente limitações relacionadas ao **hardware específico das câmeras analisadas**, especialmente a ausência de unidade de ponto flutuante (FPU) em determinadas arquiteturas, o que restringe o uso de algoritmos mais complexos, como aqueles baseados em fluxo óptico. Além disso, os experimentos são realizados apenas em dispositivos da fabricante Axis, o que pode limitar a generalização dos resultados para outras plataformas embarcadas.

Uma pergunta crítica que poderia ser feita aos autores é:

como as conclusões do estudo se aplicariam a SoCs modernos para câmeras IP, que já incorporam aceleradores de hardware para visão computacional e aprendizado de máquina?

Essa análise poderia ampliar a relevância do trabalho frente às arquiteturas embarcadas atuais.

Artigo 2:

A. C. Cob-Parro, C. Losada-Gutiérrez, M. Marrón-Romera, A. Gardel-Vicente e I. Bravo-Muñoz,

“Smart Video Surveillance System Based on Edge Computing,” *Sensors*, vol. 21, no. 9, art. 2958, 2021.

doi: 10.3390/s21092958.

Resumo: O artigo apresenta o desenvolvimento de um sistema inteligente de vigilância por vídeo baseado em computação na borda (edge computing), capaz de detectar, rastrear e contar pessoas em tempo real. O sistema executa algoritmos de inteligência artificial diretamente em dispositivos embarcados de baixo consumo energético, evitando a necessidade de processamento centralizado em servidores. A solução proposta utiliza uma arquitetura MobileNet-SSD para detecção de pessoas, combinada com um banco de filtros de Kalman para rastreamento. A plataforma embarcada selecionada inclui um processador Intel Atom e uma Vision Processing Unit (VPU) Myriad-X, responsável por acelerar a inferência de redes neurais. Resultados experimentais demonstram boa precisão, capacidade de processar múltiplos fluxos de vídeo simultaneamente e redução do consumo energético em comparação com abordagens baseadas apenas em CPU ou GPU.

Perguntas:

Qual é a questão de pesquisa principal abordada pelo artigo? Qual a importância do problema tratado para a área do conhecimento?

R: A principal questão de pesquisa do artigo é **avaliar se algoritmos de visão computacional baseados em deep learning podem ser executados de forma eficiente em dispositivos embarcados de baixo consumo, atuando como nós inteligentes em sistemas distribuídos de vigilância por vídeo**. Esse problema é extremamente relevante para a área de sistemas embarcados e Internet das Coisas (IoT), pois sistemas de vigilância tradicionais dependem de arquiteturas centralizadas, que demandam alta largura de banda, maior latência e elevado consumo energético. Ao demonstrar que é possível realizar detecção, rastreamento e contagem de pessoas diretamente no nó de borda, o estudo contribui para o avanço de soluções escaláveis, eficientes e mais adequadas a aplicações reais em ambientes urbanos, industriais e públicos.

- Os autores mencionam limitações do estudo? Se sim, quais? Que pergunta ou crítica você faria aos autores para esclarecer ou aprofundar algum ponto?

R: Os autores apontam implicitamente algumas limitações do trabalho. Uma delas está relacionada ao **uso de um hardware específico (UpSquared2 com VPU Myriad-X)**, o que pode restringir a generalização dos resultados para outros SoCs embarcados amplamente utilizados em câmeras comerciais, como plataformas ARM baseadas em SoCs dedicados de vídeo. Além disso, o sistema foi avaliado principalmente em bases de dados controladas (EPFL), que, embora representativas, não contemplam toda a variabilidade de cenários reais, como condições climáticas extremas, iluminação noturna severa ou ambientes externos complexos.

Uma questão crítica que poderia ser levantada aos autores é:

como o desempenho do sistema seria impactado em plataformas embarcadas mais simples, sem VPU dedicada, como SoCs de câmeras IP comerciais baseadas em ARM, e quais adaptações seriam necessárias para manter o processamento em tempo real? Essa análise ampliaria a aplicabilidade prática do trabalho para dispositivos amplamente disponíveis no mercado.

PARTE 3:

Processamento multitarefa: Implementar na ESP32 duas tasks por meio de FreeRTOS. Cada task deve ter um propósito específico. Poderá envolver o uso de sensores, conversor ADC e DAC, PWM, timer e interrupção, comunicação serial e sem fio via Bluetooth e Wi-Fi etc (exemplo: task1: PWM com controle de duty cycle via aplicativo móvel por Bluetooth; task2: sensor capacitivo).

Processamento multinúcleo: no contexto acima, a primeira Task deve ser configurada como alta prioridade e ser alocada no núcleo 1 da ESP32. Por outro lado, a segunda Task será de baixa prioridade e alocada no núcleo 0 da ESP32.

Parte 1: código que roda tasks em diferentes núcleos:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// =====
// TASK NO CORE 0
// =====
void TaskCore0(void *pvParameters) {
    while (1) {
        Serial.print("Rodando TASK no CORE ");
        Serial.println(xPortGetCoreID());

        vTaskDelay(pdMS_TO_TICKS(1000)); // 1 s
    }
}

// =====
// TASK NO CORE 1
// =====
void TaskCore1(void *pvParameters) {
```

```

while (1) {
    Serial.print("Rodando TASK no CORE ");
    Serial.println(xPortGetCoreID());

    vTaskDelay(pdMS_TO_TICKS(1000)); // 1 s
}

// =====
// SETUP
// =====
void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println("\n=== TESTE FREE RTOS MULTI-CORE ===");

    // Task fixada no CORE 0
    xTaskCreatePinnedToCore(
        TaskCore0,    // função
        "TaskCore0",  // nome
        2048,         // stack
        NULL,
        1,            // prioridade
        NULL,
        0             // CORE 0
    );

    // Task fixada no CORE 1
    xTaskCreatePinnedToCore(
        TaskCore1,
        "TaskCore1",
        2048,
        NULL,
        1,
        NULL,
        1             // CORE 1
    );
}

void loop() {
    // loop vazio
    vTaskDelay(pdMS_TO_TICKS(1000));
}

```

```
}
```

imprime: Rodando TASK no CORE 0

Rodando TASK no CORE 1

Rodando TASK no CORE 0

Rodando TASK no CORE 1

...

Parte 2: Blink multi núcleo: núcleo 0 pisca o led a cada 0.3s, no núcleo 1 pisca o led a cada 1.5s.

```
#include "freertos/FreeRTOS.h"
```

```
#include "freertos/task.h"
```

```
#define LED_CORE0 2 // LED do Core 0 (geralmente onboard)
```

```
#define LED_CORE1 4 // LED do Core 1
```

```
// =====
```

```
// TASK CORE 0 – 0.3 s
```

```
// =====
```

```
void TaskBlinkCore0(void *pvParameters) {
```

```
    while (1) {
```

```
        digitalWrite(LED_CORE0, !digitalRead(LED_CORE0));
```

```
        Serial.print("LED Core 0 | Core = ");
```

```
        Serial.println(xPortGetCoreID());
```

```
        vTaskDelay(pdMS_TO_TICKS(300)); // 0.3 s
```

```
    }
```

```
}
```

```
// =====
```

```
// TASK CORE 1 – 1.5 s
```

```
// =====
```

```
void TaskBlinkCore1(void *pvParameters) {
```

```
    while (1) {
```

```
        digitalWrite(LED_CORE1, !digitalRead(LED_CORE1));
```

```
        Serial.print("LED Core 1 | Core = ");
```

```
        Serial.println(xPortGetCoreID());
```

```
        vTaskDelay(pdMS_TO_TICKS(1500)); // 1.5 s
```

```
    }
```

```
}
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup() {
```

```

Serial.begin(115200);
delay(1000);

pinMode(LED_CORE0, OUTPUT);
pinMode(LED_CORE1, OUTPUT);

Serial.println("\n=== FreeRTOS Multinúcleo – Blink LEDs ===");

// Task no CORE 0
xTaskCreatePinnedToCore(
    TaskBlinkCore0,
    "BlinkCore0",
    1024,
    NULL,
    1,
    NULL,
    0
);

// Task no CORE 1
xTaskCreatePinnedToCore(
    TaskBlinkCore1,
    "BlinkCore1",
    1024,
    NULL,
    1,
    NULL,
    1
);
}

void loop() {
    // Nada aqui — tudo roda nas Tasks
    vTaskDelay(pdMS_TO_TICKS(1000));
}

```

(8,5/10) Feedback: Oi pessoal, faltou apresentar a montagem do circuito que vocês utilizaram para realizar o trabalho (-0.5) e comentar sobre a diferença entre processos/threads e FreeRTOS (-1.0).